Proc. Natl. Acad. Sci. USA
Vol. 93, pp. 1591–1595, February 1996
Applied Mathematics

A fast marching level set method for monotonically advancing fronts

J. A. SETHIAN

Department of Mathematics, University of California, Berkeley, CA 94720

Communicated by Alexandre J. Chorin, University of California, Berkeley, CA, November 16, 1995 (received for review October 20, 1995)

Downloaded from https://www.pnas.org by PENNSYLVANIA STATE UNIVERSITY PSU; ELECTRONIC RESOURCES UNIT on March 15, 2026 from IP address 128.118.12.21.

ABSTRACT **A fast marching level set method is presented for monotonically advancing fronts, which leads to an extremely fast scheme for solving the Eikonal equation. Level set methods are numerical techniques for computing the position of propagating fronts. They rely on an initial value partial differential equation for a propagating level set function and use techniques borrowed from hyperbolic conservation laws. Topological changes, corner and cusp development, and accurate determination of geometric properties such as curvature and normal direction are naturally obtained in this setting. This paper describes a particular case of such methods for interfaces whose speed depends only on local position. The technique works by coupling work on entropy conditions for interface motion, the theory of viscosity solutions for Hamilton–Jacobi equations, and fast adaptive narrow band level set methods. The technique is applicable to a variety of problems, including shape-from-shading problems, lithographic development calculations in microchip manufacturing, and arrival time problems in control theory.**

This paper describes and tests a numerical algorithm for tracking the evolution of interfaces. The technique applies in the case of a front propagating normal to itself with a speed F that depends only on position and is always either positive or negative. The applications of such a technique include some global illumination problems and problems from control theory, as well as surface advancement in lithographic development and isotropic etching and deposition in the manufacturing of microelectronic structures. This scheme was first described in ref. 1; this paper presents the details of this scheme and shows results and timings.

Background

Consider a boundary, either a curve in two dimensions or a surface in three dimensions, separating one region from another, and imagine that this curve/surface moves in its normal direction with a known speed function F. The goal is to track the motion of this interface as it evolves. We are only concerned with the motion of the interface in its normal direction and shall ignore tangential motion.

As shown in refs. 2, 3, and 4, a propagating interface can develop corners and discontinuities as it evolves, which require the introduction of a weak solution in order to proceed. The correct weak solution comes from enforcing an entropy condition for the propagating interface, similar to the one in gas dynamics. Furthermore, this entropy-satisfying weak solution is the one obtained by considering the limit of smooth solutions for the problem in which curvature plays a regularizing role.

As an example, consider the initial cosine curve propagating with speed $F = 1$ shown in Fig. 1. As the front moves, a corner forms in the propagating front, which corresponds to a shock in the slope, and a weak solution must be developed beyond this point. If the motion of each individual point is continued, the result is the swallowtail solution shown in Fig. 1*A*, which is multiple-valued and does not correspond to a clear interface separating two regions. Instead, an appropriate weak solution is obtained by considering the associated smooth flow obtained by adding curvature κ to the speed law—that is, letting $F = 1 - \varepsilon\kappa$ (see Fig. 1*B*). The limit of these smooth solutions as ε goes to zero produces the weak solution shown in Fig. 1*C*; this is the same solution obtained by enforcing an entropy condition, similar to the one for a scalar hyperbolic conservation law, which selects the envelope obtained by Huygens principle as the correct solution (see ref. 2). This weak solution corresponds to a decrease in total variation of the propagating front and is irreversible (3). For details, see ref. 3.

As a numerical technique, this suggests using the technology from hyperbolic conservation laws to solve the equations of motion, as described in ref. 5. This leads to the "level set" formulation introduced in ref. 6, which we now describe.

Level Set Methods. Given an initial position for an interface Γ, where Γ is a closed curve in R^2, and a speed function F, which gives the speed of Γ in its normal direction, the level set method takes the perspective of viewing Γ as the zero level set of a function $\phi(x, t = 0)$ from R^2 to R. That is, let $\phi(x, t = 0) = \pm d$, where d is the distance from x to Γ, and the plus (minus) sign is chosen if the point x is outside (inside) the initial hypersurface Γ. Then, by the chain rule, an evolution equation for the interface may be produced (4, 6)—namely,

$$\phi_t + F|\nabla\phi| = 0, \qquad [1]$$

$$\phi(x, t = 0) = given. \qquad [2]$$

This is an initial value partial differential equation in one higher dimension than the original problem. In Fig. 2 (taken from ref. 7), we show the outward propagation of an initial curve and the accompanying motion of the level set function ϕ.

There are several advantages to this level set perspective:

1. Although $\phi(x, t)$ remains a function, the level surface $\phi = 0$ corresponding to the propagating hypersurface may change topology, as well as form sharp corners as ϕ evolves (see ref. 6).
2. Second, a discrete grid can be used together with finite differences to devise a numerical scheme to approximate the solution. Care must taken to adequately account for the spatial derivatives in the gradient.
3. Third, intrinsic geometric properties of the front are easily determined from the level set function ϕ. The normal vector is given by $\vec{n} = \nabla\phi/|\nabla\phi|$ and the curvature of each level set is $\kappa = \nabla\cdot\nabla\phi/|\nabla\phi|$.
4. Finally, the formulation is unchanged for propagating interfaces in three dimensions.

Since its introduction in ref. 6, the above level set approach has been used in a wide collection of problems involving moving interfaces. Some of these applications include the generation of minimal surfaces (8), singularities and geodesics in moving curves and surfaces in ref. 9, flame propagation (10, 11), fluid interfaces (12, 13), shape reconstruction (14,

The publication costs of this article were defrayed in part by page charge payment. This article must therefore be hereby marked "*advertisement*" in accordance with 18 U.S.C. §1734 solely to indicate this fact.

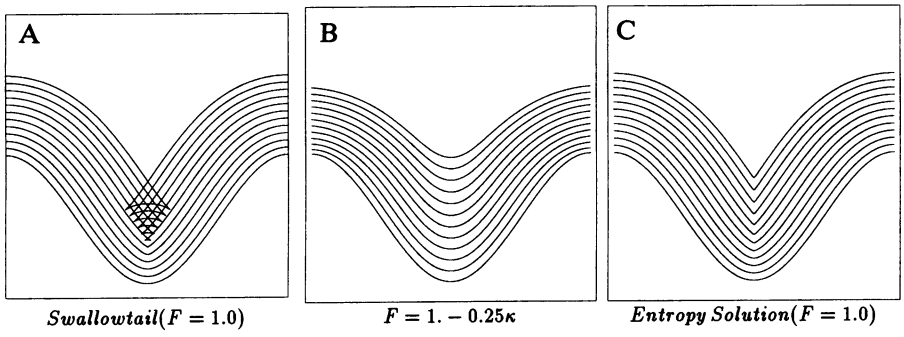

FIG. 1. Cosine curve propagating with unit speed.

15), as well as etching, deposition, and lithography calculations in refs. 16 and 17. Extensions of the basic technique include fast methods in ref. 18, level set techniques for multiple fluid interfaces and triple point junctions in ref. 19, and grid generation in ref. 7. The fundamental Eulerian perspective presented by this approach has since been adopted in many theoretical analyses of mean curvature flow (in particular, see refs. 20 and 21).

Numerical Approximation. As mentioned above, a careful approximation to the gradient in the level set equation (Eq. **1**) is required to produce the correct weak solution. One of the simplest such schemes is given in ref. 6—namely,

$$\begin{aligned}\phi_{ij}^{n+1} = \phi_{ij}^{n} &- \Delta t(\max(D_{ij}^{-x}\phi, 0)^2 \\ &+ \min(D_{ij}^{+x}\phi, 0)^2 \max(D_{ij}^{-y}\phi, 0)^2 \\ &+ \min(D_{ij}^{+y}\phi, 0)^2)^{1/2}, \qquad [3]\end{aligned}$$

where the speed is $F = 1$ and difference operator notation is employed; for example, $D_{ij}^{+x}\phi = (\phi_{i+1,j} - \phi_{i,j})/(\Delta x)$. The crucial point in this (any such appropriate) numerical scheme is the correct direction of the upwinding and treatment of sonic points.

Narrow Band Methods. The above technique relys on computing the evolution of *all* the level sets, not simply the zero level set corresponding to the front itself. As such, it is a computationally expensive technique, since an extra dimension has been added to the problem.

As an alternative, an efficient modification is to perform work only in a neighborhood of the zero level set; this is known as the "narrow band approach." In this case, the operation count in three dimensions for N^3 grid points drops to $O(kN^2)$, where k is the number of cells in the width of the narrow band, providing a significant cost reduction. This narrow band method method was introduced in ref. 8, used in recovering shapes from images in ref. 14, and analyzed extensively in ref. 18.

The basic idea is to tag grid points as either "alive," "land mines," or "far away," depending on whether they are inside the band, near its boundary, or outside the band, respectively (see Fig. 3). Thus, work is performed only on the alive points, and the band is reconstructed once land mine points are reached. An extreme one-cell version of this leads to the fast marching level set method presented below.

A Fast Marching Level Set Method

We now discuss in detail the fast marching level set method introduced in ref. 1. Consider the special case of a front moving with speed $F = F(x, y, z), F > 0$ (the case where F is everywhere negative is also allowed). We then have a monotonically advancing front whose level set equation is of the form

$$\phi_t + F(x, y, z)|\nabla_\phi| = 0 \qquad [4]$$

$$\phi(x, t = 0) = \Gamma. \qquad [5]$$

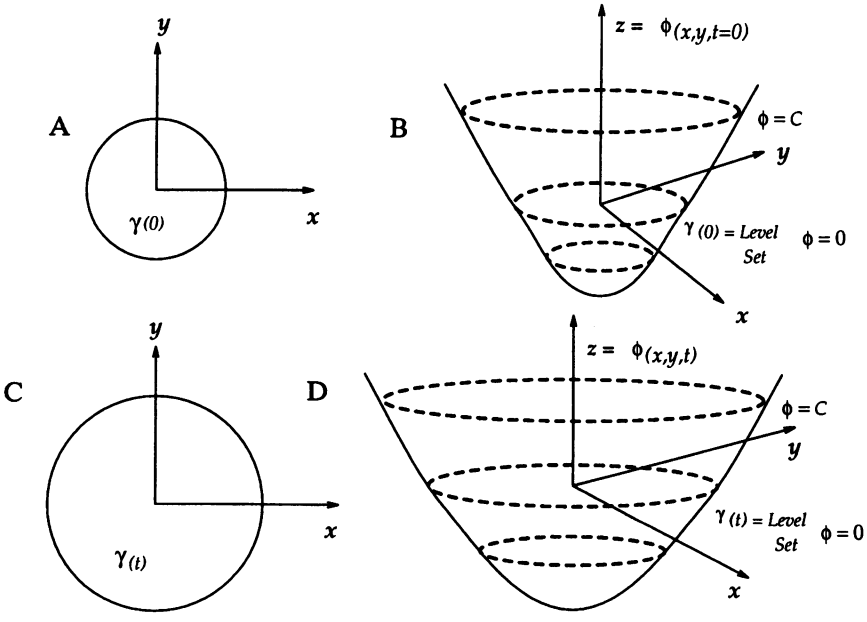

FIG. 2. Propagating circle. [Reprinted with permission from ref. 7 (Springer).]

Downloaded from https://www.pnas.org by PENNSYLVANIA STATE UNIVERSITY PSU; ELECTRONIC RESOURCES UNIT on March 15, 2026 from IP address 128.118.12.21.

Downloaded from https://www.pnas.org by PENNSYLVANIA STATE UNIVERSITY PSU; ELECTRONIC RESOURCES UNIT on March 15, 2026 from IP address 128.118.12.21.

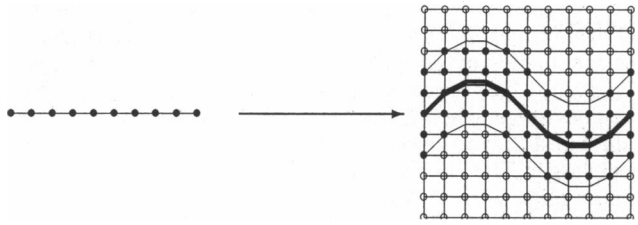

FIG. 3. Pointer array tags alive, narrow band, and far away points.

Imagine the two-dimensional case in which the interface is a propagating curve, and suppose we graph the evolving zero level set above the xy plane. That is, let $T(x, y)$ be the time at which the curve crosses the point (x,y). The surface $T(x,y)$ then satisfies the equation

$$|\nabla T|F = 1. \quad [6]$$

Eq. **6** simply says that the gradient of arrival time surface is inversely proportional to the speed of the front.

This is a form of the well-known Eikonal equation, and the recasting of the problem into a stationary one is common in a variety of applications. The notion of viscosity solutions is intimately connected to this equation; a central idea, discussed in detail in ref. 22, is that the use of monotone, consistent schemes will lead to schemes that select the correct viscous limit of the partial differential equation, as was done in the level set scheme in ref. 6. We refer the interested reader to large literature on this subject, including relevant theory in refs. 22–25 and numerical algorithms in refs. 26–29. Roughly speaking, two possible ways to view these solution techniques are either iteration toward the solution or direct construction of the stationary solution surface $T(x, y)$. We now discuss a technique that relies on the level set methodology and narrow band work described above.

Approximation Scheme. For this discussion, we limit ourselves to a two-dimensional problem inside a square from [0, 1] × [0, 1] and imagine that the initial front is along the line $y = 0$; furthermore, we assume that we are given a positive speed function $F(x, y)$ that is periodic in x. Thus, the front propagates upward off the initial line. Using our approximation to the gradient, we are then looking for a solution in the unit box to the equation

$$[\max(D_{ij}^{-x}T, 0)^2 + \min(D_{ij}^{+x}T, 0)^2 + \max(D_{ij}^{-y}T, 0)^2 + \min(D_{ij}^{+y}T, 0)^2)] = \frac{1}{F^2}, \quad [7]$$

where $T(x, 0) = 0$.

Since Eq. **7** is in essence a quadratic equation for the value at each grid point (assuming the others are held fixed), one typically iterates until convergence by solving the equation at each grid point, selecting the largest possible value as the solution in accordance with obtaining the correct viscosity solution. An iterative algorithm for computing the solution to this problem was introduced by Rouy and Tourin (29); there, a different approximation to the gradient was chosen, which is less diffusive, namely,

$$[\max(\max(D_{ij}^{-x}T, 0), -\min(D_{ij}^{+x}T, 0))^2 + \max(\max(D_{ij}^{-y}T, 0), -\min(D_{ij}^{+y}T, 0))^2] = 1/F_{ij}^2. \quad [8]$$

For details of this approach, see ref. 29. This approximation to the gradient will be used in the fast marching level set method.

A Fast Marching Level Set Method. The key to constructing a fast marching algorithm is the observation that the upwind difference structure of Eq. **8** means that information propagates "one way"—that is, from smaller values of T to larger values. Hence, our algorithm rests on "solving" Eq. **8** by building the solution outward from the smallest time value T. The idea is to sweep the front ahead in an upwind fashion by considering a set of points in narrow band around the existing front and to march this narrow band forward, freezing the values of existing points and bringing new ones into the narrow band structure. The key is in the selection of *which* grid point in the narrow band to update. The technique is easiest to explain algorithmically (see Fig. 4, taken from ref. 1). We imagine that we want to propagate a front upward through an N by N grid with speed F_{ij} giving the speed in the normal direction at each grid point. Here the set of grid points $j = 1$ correspond to the y axis, and we assume that $F_{ij} > 0$.

1. Initialize
   (a) (Alive points: shaded points): Let A be the set of all grid points $\{i, j = 1\}$; set $T_{i,1} = 0.0$ for all points in A.
   (b) (Narrow band points: circles): Let *Narrow Band* be the set of all grid points $\{i, j = 2\}$; set $T_{i,1} = dy/F_{ij}$ for all points in *Narrow Band.*
   (c) (Far away points: rectangles): Let *Far Away* be the set of all grid points $\{i, j > 2\}$; set $T_{i,j} = \infty$ for all points in *Far Away.*
2. Marching Forward
   (a) Begin Loop: Let $(i_{\min}, j_{\min})$ be the point in *Narrow Band* with the smallest value for T.
   (b) Add the point $(i_{\min}, j_{\min})$ to A; remove it from *Narrow Band.*
   (c) Tag as neighbors any points $(i_{\min}, -1, j_{\min})$, $(i_{\min} + 1, j_{\min})$, $(i_{\min}, j_{\min} - 1)$, $(i_{\min}, j_{\min} + 1)$ that are either in *Narrow Band* or *Far Away.* If the neighbor is in *Far Away*, remove it from that list and add it to the set *Narrow Band.*
   (d) Recompute the values of T at all neighbors according to Eq. **8**, selecting the largest possible solution to the quadratic equation.
   (e) Return to top of Loop.

We take periodic boundary conditions where required. Assuming for the moment that it takes no work to determine the member of the narrow band with the smallest value of T, the total work required to compute the solution at all grid points is $O(N^2)$, where calculation is performed on an N by N grid.

Why does the above algorithm work? Since we are always locating the smallest value in the narrow band, its value for T must be correct; other narrow band points or far away points with larger T values cannot affect it. The process of recomputing the T values at neighboring points (that have not been previously accepted) cannot yield a value smaller than any of that at any of the accepted points, since the correct viscosity solution is obtained by selecting the *largest* possible solution to the quadratic equation. Thus, the algorithm marches the solution outward, always selecting the narrow band grid point with minimum trial value for T and readjusting neighbors. Another way to look at this is that each minimum trial value begins an application of Huygen's principle, and the expanding wave front touches and updates all others.

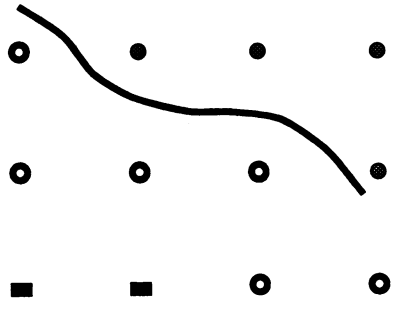

FIG. 4. Narrow band approach to marching level set method.

Downloaded from https://www.pnas.org by PENNSYLVANIA STATE UNIVERSITY PSU; ELECTRONIC RESOURCES UNIT on March 15, 2026 from IP address 128.118.12.21.

	B	
A	?	C
	D	

FIG. 5. Matrix of neighboring values.

Proof That the Algorithm Constructs a Viable Solution

Here, it is shown that the above algorithm produces a solution that everywhere satisfies the discrete version of the equation, which is given by

$$[\max(\max(D_{ij}^{-x}T, 0), -\min(D_{ij}^{+x}T, 0))^2 + \max(\max(D_{ij}^{-y}T, 0), -\min(D_{ij}^{+y}T, 0))^2] = f_{ij}^2, \quad [9]$$

where $f_{ij}^2 = 1/F_{ij}^2$. A constructive proof is given. Since the values of $T(x, y, z)$ are built by marching forward from the smallest value to the largest, one need only show that whenever a "trial" value is converted into an alive value, none of the recomputed neighbors obtain new values less than the accepted value. If this is true, then we will always be marching ahead in time, and the thus the correct "upwind" nature of the differencing will be respected. We shall prove our result in two dimensions; the three-dimensional proof is the same.

Thus, consider the matrix of grid values given in Fig. 5. The argument will follow the computation of the new value of T in the center grid point to replace the value of ?, based on the neighboring values. Assume, without loss of generality, that the value A at the left grid point is the smallest of all trial values. It is now shown that the value at the center grid point (called $T_{\text{recomputed-from-A}}$) cannot be less than A. This will prove that the upwinding is respected and that there is no need to go back and readjust previously set values. We shall consider the four cases that (1) none of the neighbors B, C, or D are alive, (2) one of these neighbors is alive, (3) two of the neighbors are alive, and (4) all three of these neighbors are alive.*

Case 1: A, B, C, and D are trial; A is the smallest. In this case, all of the neighbors around the center grid point are either trial or set to *Far Away*. Since A is the smallest such value, we convert that value to alive and recompute the value at the center grid point. We now show that the recomputed value $A \leq T_{\text{recomputed-from-A}} \leq A + f$.

1. Suppose $A + f \leq \min(B, D)$. Then $T_{\text{recomputed-from-A}} = (A + f)$ is a solution to the problem, since only the difference operator to the left grid point is nonzero. We are absorbing the grid size Δx into the inverse speed function f.
2. Suppose $A + f \geq \min(B, D)$. Then, without loss of generality, assume that $B \leq D$. We can solve the quadratic equation

$$(T_{\text{recomputed-from-A}} - A)^2 + (T_{\text{recomputed-from-A}} - B)^2 = f^2. \quad [10]$$

The discriminant is nonnegative when $f \geq (B - A)/\sqrt{2}$, which must be true since we assumed that $A + f \geq B$ and hence $f \geq (B - A)$. Thus, a solution exists, and it is easy to check that this solution must then be greater than or equal to B and thus falls into the required range. Furthermore, we see that $T \leq A + f$, since the second term on the left is nonnegative.

Thus, we have shown that $A \leq T_{\text{recomputed-from-A}} \leq A + f$, and therefore $T_{\text{recomputed-from-A}}$ cannot be less than the just converted value A.

This case will act as a template for the other cases.

Case 2: B is alive; A, C, and D are trial; A is the smallest of the trial values. In this case, A has just been converted, since it is the smallest of the trial values. It can be shown that the new $T_{\text{recomputed}_A}$ has a new value still greater than A. At some previous stage, when B was converted from trial to alive, the values of A, C, and D were all trial values and hence must have been larger. Then this means that when B was converted from trial to alive, we had the previous case above, and hence $B \leq T_{\text{recomputed-from-B}} \leq B + f$; furthermore, since the value at the center was *not* chosen as the smallest trial value, we must have that $A \leq B + f$. By the above case, we then have that $B \leq A \leq T_{\text{recomputed-from-A}} \leq B + f$, and hence the recomputed value cannot be less than the just converted value of A.

Case 3: C is alive; A, B, and D are trial; A is the smallest of the trial values. In this case, due to the direction of the upwind differencing, the value at C is the contributor in the x direction, the acceptance of A does not affect the recomputation, and the case defaults into the first case above.

The remaining cases are all the same, since the differencing takes the smallest values in each coordinate direction. The proof in three dimensions is identical.

Finding the Smallest Value. The key to an efficient version of the above technique lies in a fast way of locating the grid point in the narrow band with the smallest value for T. We use a variation on a heapsort algorithm with back pointers (see refs. 12 and 30). In more detail, imagine that the list of narrow band points is initially sorted in a heapsort so that the smallest member can be easily located. We store the values of these points in the heapsort, together with their indices, which give their location in the grid structure. We keep a companion array that points from the two-dimensional grid to the location of that grid point in the heapsort array. Finding the smallest value is easy. To find the neighbors of that point, we use the pointers from the grid array to the heapsort structure. The values of the neighbors are then recomputed, and then the results are bubbled upward in the heapsort until they reach their correct locations, at the same time readjusting the pointers in the grid array. This results in an $O(\log N)$ algorithm for the total amount of work, where N is the number of points in the narrow band.

Arbitrary Initial Fronts. The above technique considered a flat initial interface for which trial values at the narrow band points could be easily initialized. Suppose we are given an arbitrary closed curve or surface as the initial location of the front. In this case, we use the original narrow band level set method to initialize the problem. First, label all grid points as far away and assign them T values of ∞. Then, construct the signed distance function in a one-grid cell wide band around the initial hypersurface Γ. Propagate that surface both forward and backward in time until a layer of grid points is

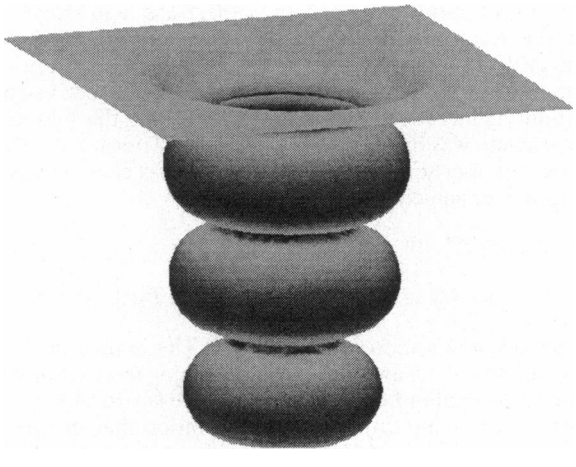

FIG. 6. Lithographic development on 50 × 50 × 50 grid.

*Recall that alive means that their T values are less than A. Here, we are using the notation that the symbol A stands for both the grid point and its T value.

Table 1. Timings for development to $T = 10$ on a Sparc 10 workstation

Grid size	Time to load rate file, sec	Time to propagate front, sec	Total time, sec
50 × 50 × 50	0.1	0.7	0.8
100 × 100 × 100	1.2	8.2	9.4
150 × 150 × 150	3.9	37.8	41.7
200 × 200 × 200	9.0	80.0	89

crossed in each direction, computing the signed crossing times as in ref. 7. Then collect the points with negative crossing times as alive points with T value equal to the crossing time and the points with positive crossing times as narrow band points with T value equal to the positive crossing times. Then begin the fast marching algorithm.

Results

Simple Initial Front. As a first example, we use the above algorithm to compute a lithographic development profile for an evolving front. We start with a flat profile at height $z = 1$ in the unit cube centered at (0.5, 0.5, 0.5) and follow the evolution of the interface downward with speed given by the model Gaussian rate function

$$F(x, y, z) = e^{-64(r^2)}(\cos^2(12z) + 0.01), \qquad [11]$$

where $r = \sqrt{(x - 0.5)^2 + (y - 0.5)^2}$. This rate function F corresponds to effect of standing waves, which change the resist properties of the material and cause sharp undulations and turns in the evolving profile. In Fig. 6, we show the profile etched out by such an initial state; the calculation is carried out until $T = 10$.

In Table 1, we give timings for a parameter study on a Sparc 10 workstation for the speed function $F = e^{-64(r^2)}(\cos^2(6z) + 0.01)$. We note that loading the file containing the model Gaussian rate function F is a significant proportion of the total compute time.

Deposition Problem

Next, we consider the case of simple isotropic deposition above a trench, with corresponding speed function $F = 1$. Fig. 7 shows a two-dimensional trench being filled in with a deposition layer; note the sharp corner that develops when the entropy condition is invoked.

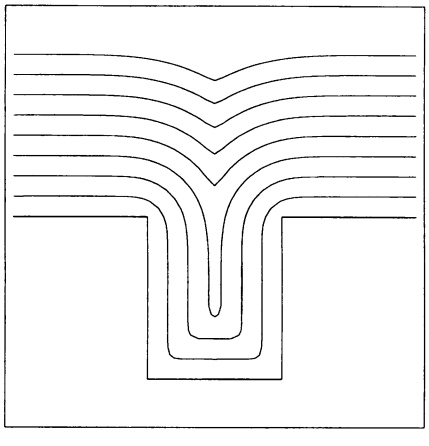

FIG. 7. Isotropic deposition above trench.

There are a large number of applications of this fast marching level set method, including problems in control theory, etching/deposition/lithography, and global illumination. At the same time, the above technique can be extended, with modification, to general convex speeds laws and, perhaps, nonconvex speed functions. We shall report on these issues elsewhere.

I thank D. Adalsteinsson and R. Malladi for valuable discussions and help with the relevant sorting algorithms. All calculations were performed at the University of California at Berkeley and the Lawrence Berkeley Laboratory. This work was supported in part by the Applied Mathematics Subprogram of the Office of Energy Research under DE-AC03-76SF00098 and the National Science Foundation Advanced Research Projects Agency under Grant DMS-8919074.

1. Sethian, J. A. (1995) *Acta Numerica*, in press.
2. Sethian, J. A. (1982) Ph.D. Dissertation (Univ. of California, Berkeley).
3. Sethian, J. A. (1985) *Comm. Math. Phys.* **101,** 487–499.
4. Sethian, J. A. (1990) *J. Differ. Geom.* **31,** 131–161.
5. Sethian, J. A. (1987) in *Variational Methods for Free Surface Interfaces*, eds. Concus, P. & Finn, R. (Springer, New York), pp. 155–164.
6. Osher, S. & Sethian, J. A. (1988) *J. Comput. Phys.* **79,** 12–49.
7. Sethian, J. A. (1994) *J. Comp. Phys.* **115,** 440–454.
8. Chopp, D. L. (1993) *J. Comp. Phys.* **106,** 77–91.
9. Chopp, D. L. & Sethian, J. A. (1993) *J. Exp. Math.* **2,** 235–255.
10. Rhee, C., Talbot, L. & Sethian, J. A. (1995) *J. Fluid Mech.* **300,** 87–115.
11. Zhu, J. & Sethian, J. A. (1992) *J. Comp. Phys.* **102,** 128–138.
12. Press, W. H. (1988) *Numerical Recipes* (Cambridge Univ. Press, New York).
13. Mulder, W., Osher, S. J. & Sethian, J. A. (1992) *J. Comp. Phys.* **100,** 209–228.
14. Malladi, R., Sethian, J. A. & Vemuri, B. C. (1994) *Lect. Notes Computer Sci.* **800,** 3–13.
15. Malladi, R. & Sethian, J. A. (1995) *Proc. Natl. Acad. Sci. USA* **92,** 7046–7050.
16. Adalsteinsson, D. & Sethian, J. A. (1995) *J. Comp. Phys.* **120,** 128–144.
17. Adalsteinsson, D. & Sethian, J. A. (1995) *J. Comp. Phys.* **122,** 348–366.
18. Adalsteinsson, D. & Sethian, J. A. (1995) *J. Comp. Phys.* **118,** 269–277.
19. Sethian, J. A. (1995) in *Computational Fluid Dynamics Reviews*, eds. Oshima, K. & Hafez, M. (Wiley, New York).
20. Chen, Y., Giga, Y. & Goto, S. (1991) *J. Differ. Geom.* **33,** 749.
21. Evans, L. C. & Spruck, J. (1991) *J. Differ. Geom.* **33,** 635.
22. Lions, P. L. (1982) *Generalized Solution of Hamilton-Jacobi Equations* (Pittman, London).
23. Barles, G. & Souganidis, P. E. (1991) *Asymptotic Anal.* **4,** 271–283.
24. Crandall, M. G., Evans, L. C. & Lions, P.-L. (1984) *Trans. Am. Math. Soc.* **282,** 487–502.
25. Souganidis, P. E. (1985) *J. Differ. Equ.* **59,** 1–43.
26. Bardi, M. & Falcone, M. (1990) *SIAM J. Control Optim.* **28,** 950–965.
27. Falcone, M., Giorgi, T. & Loretti, P. (1994) *SIAM J. Appl. Math.* **54,** 1335–1354.
28. Kimmel, R. & Bruckstein, A. (1992) *Shape from Shading via Level Sets*, Center for Intelligent Systems Rep. 9209 (Technion–Israel Institute of Technology, Haifi, Israel).
29. Rouy, E. & Tourin, A. (1992) *SIAM J. Num. Anal.* **29,** 867–884.
30. Sedgewick, R. (1988) *Algorithms* (Addison–Wesley, Reading, MA).

Downloaded from https://www.pnas.org by PENNSYLVANIA STATE UNIVERSITY PSU; ELECTRONIC RESOURCES UNIT on March 15, 2026 from IP address 128.118.12.21.